



FACULTY OF ENGINEERING

AIN SHAMS UNIVERSITY

CSE211s – Introduction to Embedded Systems | Spring 2026

Object-Following Robot Car

Using STM32 Nucleo Board & HC-SR04 Ultrasonic Sensor

Final Project Report

Group 3 – Section 1

Group 3 – Section 1	
Mahmoud Hesham Mohamed	2100605
Fatma Samy Ahmed Khalil	2100660
Yousef Ahmed Nabil	2200628
Abedallah Mohammed Fayyad	2201315
Mohamed Samer Mohamed	2100905

Table of Contents :

1. INTRODUCTION.....	3
2.HARDWARECOMPONENTS.....	3
2.1 STM32 NUCLEO BOARD.....	3
2.2 HC-SR04 ULTRASONIC SENSOR.....	3
2.3 ROBOT CAR CHASSIS.....	3
3. SOFTWARE STRUCTURE.....	3
3.1 STARTUP CODE & INITIALIZATION.....	3
3.2 INTERRUPT SERVICE ROUTINES (ISRS).....	4
3.3 ULTRASONIC TRIGGER.....	4
3.4 MOTOR CONTROL FUNCTIONS.....	4
3.5 DISTANCE DECISION LOGIC.....	5
3.6 MAIN CONTROL LOOP.....	5
4. SYSTEM BEHAVIOUR SUMMARY.....	5
5. PIN ASSIGNMENT.....	6
6. FULL SOURCE CODE.....	7
7. CONCLUSION.....	8
8. PROJECT VIDEO.....	8

1. INTRODUCTION

This report documents the design and implementation of an object-following robot car developed for the CSE211s Introduction to Embedded Systems course. The system uses an STM32 Nucleo development board paired with an HC-SR04 ultrasonic sensor to detect the distance of a moving object and autonomously control a four-wheel drive robot chassis to follow it while maintaining a safe standoff distance. The project demonstrates practical application of embedded systems concepts including timer-based interrupt service routines (ISRs), PWM motor control, and sensor interfacing.

2. HARDWARE COMPONENTS

2.1 STM32 NUCLEO BOARD

The STM32 Nucleo board serves as the main microcontroller unit. It runs the mbed OS RTOS framework and is responsible for:

- Generating the ultrasonic trigger pulse on pin D6
- Capturing the echo response on pin D7 via external interrupts
- Driving the motor H-bridge direction pins (D4, D5)
- Generating PWM signals for motor speed control on pins D3 and D9

2.2 HC-SR04 ULTRASONIC SENSOR

The HC-SR04 is a low-cost ultrasonic distance sensor with a measuring range of approximately 2 cm to 400 cm and a resolution of about 3 mm. It operates as follows:

- A 10 μ s HIGH pulse is sent to the TRIG pin to initiate a measurement.
- The sensor emits eight 40 kHz ultrasonic bursts and listens for the echo.
- The ECHO pin goes HIGH for a duration proportional to the round-trip travel time of the sound wave.
- Distance is calculated as: $\text{Distance (cm)} = \text{Echo pulse width } (\mu\text{s}) \times 0.0343 / 2$

2.3 ROBOT CAR CHASSIS

A four-wheel drive chassis is used with four DC motors controlled through an L298N H-bridge motor driver. The control pins (IN2, IN4) set the motor direction forward, while PWM duty cycle controls the speed in (EN A, EN B).

3. SOFTWARE STRUCTURE

The firmware is written in C++ using the mbed OS API and consists of three logical layers: startup/initialization, the main control loop, and interrupt service routines (ISRs).

3.1 STARTUP CODE & INITIALIZATION

Before the main loop begins, the following initialization steps are performed:

- InterruptIn echo(D7) – registers rising and falling edge ISR callbacks on the ECHO pin.
- PWM Out motor Speed(D3) and PWM Out motorSpeed1(D9) – PWM outputs initialized with a 1 kHz period (0.001 s).
- Timer timer – a microsecond-resolution hardware timer used to measure echo pulse width.
- All motor outputs are configured (Digital Out in2(D4), Digital Out in4(D5)).

```
echo.rise(&echo_rise);
echo.fall(&echo_fall);
motorSpeed.period(0.001f);
motorSpeed1.period(0.001f);
```

3.2 INTERRUPT SERVICE ROUTINES (ISRS)

Two ISRs handle the timing of the ultrasonic echo pulse:

- **ECHO_RISE()** – triggered on the rising edge of ECHO. Resets and starts the hardware timer.

```
void echo_rise() {
    timer.reset();
    timer.start();
}
```

- **ECHO_FALL()** – triggered on the falling edge of ECHO. Stops the timer and computes the distance in centimetres using the speed of sound at room temperature (343 m/s = 0.0343 cm/μs).

```
void echo_fall() {
    timer.stop();
    distance0 = (timer.elapsed_time().count() * 0.0343f) / 2.0f;
}
```

The volatile qualifier on distance0 ensures the compiler does not cache the value in a register across the ISR boundary.

3.3 ULTRASONIC TRIGGER

The ultrasonic_trigger() function generates the required 10 μs pulse on the TRIG pin:

```
void ultrasonic_trigger() {
    trig = 0;
    wait_us(2);
    trig = 1;
    wait_us(10);
    trig = 0;
}
```

3.4 MOTOR CONTROL FUNCTIONS

Two helper functions abstract the motor control logic:

- **MOTOR_STOP()** – drives both motors at zero duty cycle (robot halts).
- **MOTOR_FORWARD()** – drives both motors at 32.5% duty cycle (moderate following speed).

3.5 DISTANCE DECISION LOGIC

The `should_move()` function encodes the following/stopping behaviour based on two thresholds:

- **MAX_DISTANCE = 55 cm** – if the object is farther than this, move forward to catch up.
- **MIN_DISTANCE = 30 cm** – if the object is closer than this, stop to avoid collision.
- **Between 30 cm and 55 cm** – continue moving forward to maintain following distance.

3.6 MAIN CONTROL LOOP

The `main()` function runs an infinite loop that: (1) triggers a measurement, (2) evaluates the distance, (3) commands the motors, and (4) waits 100 ms before the next cycle.

```
int main() {
    echo.rise(&echo_rise);
    echo.fall(&echo_fall);
    motorSpeed.period(0.001f);
    motorSpeedl.period(0.001f);
    while (true) {
        ultrasonic_trigger();
        if (should_move(distance0))
            motor_forward();
        else
            motor_stop();
        ThisThread::sleep_for(100ms);
    }
}
```

4. SYSTEM BEHAVIOUR SUMMARY

Distance (cm)	Motor State	Description
$d \leq 30$	STOP	Object too close – halt to prevent collision

Distance (cm)	Motor State	Description
$30 < d \leq 55$	FORWARD (32.5%)	Object in safe following range – continue at moderate speed
> 55	FORWARD (32.5%)	Object too far – move forward to close the gap

5. PIN ASSIGNMENT

Nucleo Pin	Signal	Connected To
D6	TRIG (Digital Out)	HC-SR04 TRIG pin
D7	ECHO (InterruptIn)	HC-SR04 ECHO pin
D4	IN1 (Digital Out)	Motor driver direction pin 2
D5	IN2 (Digital Out)	Motor driver direction pin 4
D3	Motor Speed (PWM Out)	Motor driver PWM (left motors)
D9	Motor Speed1 (PWM Out)	Motor driver PWM (right motors)

6. FULL SOURCE CODE

The complete annotated firmware is provided below:

```
#include "mbed.h"

DigitalOut  trig(D6);
InterruptIn echo(D7);
DigitalOut  in1(D4);
DigitalOut  in2(D5);
PwmOut      motorSpeed(D3);
PwmOut      motorSpeed1(D9);
Timer timer;
#define MAX_DISTANCE 55
#define MIN_DISTANCE 30
volatile float distance0 = 0.0f;
void echo_rise() {
    timer.reset();
    timer.start();
}
void echo_fall() {
    timer.stop();
    distance0 = (timer.elapsed_time().count() * 0.0343f) / 2.0f;
}
void ultrasonic_trigger() {
    trig = 0; wait_us(2);
    trig = 1; wait_us(10);
    trig = 0;
}
void motor_stop() {
    in1 = 1; in2 = 1;
    motorSpeed.write(0);
    motorSpeed1.write(0);
}

void motor_forward() {
    in1 = 1; in2 = 1;
    motorSpeed.write(0.325f); // 32.5% duty cycle
    motorSpeed1.write(0.325f);
}

bool should_move(float d) {
```

```

    if (d > MAX_DISTANCE) return true;
    if (d <= MIN_DISTANCE) return false;
    return true;
}

int main() {
    echo.rise(&echo_rise);
    echo.fall(&echo_fall);
    motorSpeed.period(0.001f);
    motorSpeed1.period(0.001f);

    while (true) {
        ultrasonic_trigger();
        if (should_move(distance0))
            motor_forward();
        else
            motor_stop();
        ThisThread::sleep_for(100ms);
    }
}

```

7. CONCLUSION

The project successfully demonstrates a real-time embedded control loop on the STM32 Nucleo platform. Key design outcomes include:

- Interrupt-driven ultrasonic sensing eliminates busy-wait polling and keeps the main thread responsive.
- PWM motor control allows smooth speed adjustment without hardware relay switching.
- A simple two-threshold hysteresis in `should_move()` prevents rapid oscillation between stop and move states.
- The mbed OS API (InterruptIn, PwmOut, Timer, ThisThread) provides hardware abstraction and portability across STM32 targets.

Possible future improvements include PID-based speed control for smoother following behaviour, left/right steering for curved paths, and an OLED display for live distance readout.

8. PROJECT VIDEO

https://drive.google.com/drive/folders/1mUJnBdD9e2O7In-maTfzr3B0_-KyYGPp?usp=drive_link